AMATH 515: OPTIMIZATION ALGORITHMS

PARTHA KRISHNAMURTHY

Applied And Computational Mathematics, University of Washington, Seattle, WA
ipartha2@uw.edu

1. Introduction

I am comparing the computational cost of different optimization algorithms. In order to truly test the algorithm's efficiency, I picked one non-convex (Rosenbrock) and one convex (Logistic Regression) function.

2. Methods

2.1. Determining Step size: Line search using bisection.

This method performs a bisection to determine the right step size, α, that satisfies the Armijo and the curvature conditions. The search interval is initialized between 0 and inf and is adjusted based on the function and gradient violating the Wolfe conditions.

2.2. Optimization algorithms.

The general form of descent algorithms is given by,

$$x^{k+1} = x^k + \alpha^k d^k \text{ where } d^k = -B(x^k)\nabla f(x^k)$$

In this paper I compare four different ways to compute the matrix $B(x^k)$, while using line search to determine the step size. In all the methods, the convergence is determined by norm of the gradient going below a configurable tolerance value.

2.2.1. *Steepest descent.* In this method, I only rely on gradient to take the next step in iteration. I achieved this by setting $B(x^k)$ to be 1.

2.2.2. *Newton's method.* Newton's method uses 2nd order information to determine the next step. At every step, I calculate the Hessian, ∇^2, and take the dot product of its inverse with the gradient.

2.2.3. *DFP.* This is a quasi-Newton method where I approximate the inverse of Hessian using rank two updates. The gradient from the previous step is used and the current step is adjusted by subtracting the correction term.

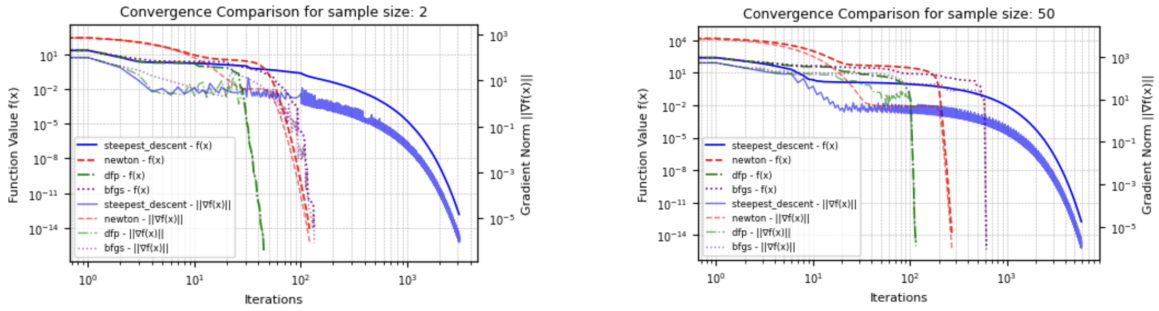
2.2.4. *BFGS.* This is another quasi-Newton method where I approximate the Hessian. The gradient from the previous step is used and the current step is adjusted by taking the dot product with Hessian approximation of current step.

3. Results

3.1. Rosenbrock function.

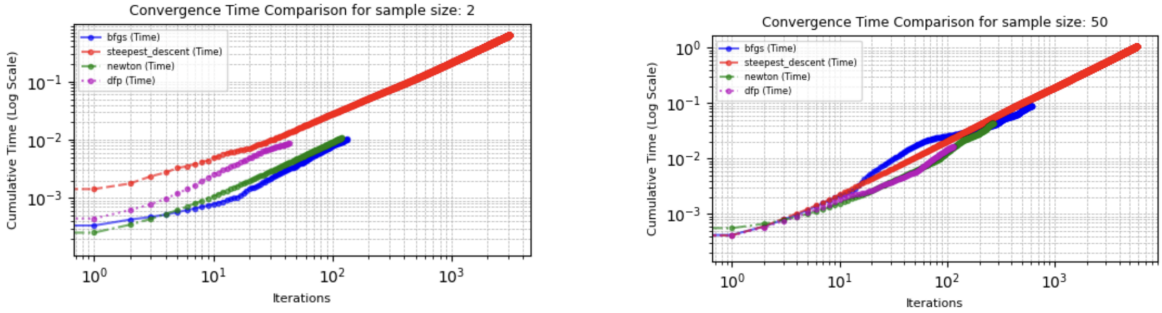
This function was tested using the 4 optimization methods with sample sizes of 2, 5, 10 and 50.

Date: /16/2025.



(A) sample size: 2 (B) sample size: 50

FIGURE 1. Newton based methods consistently perform better compared to gradient based methods on Rosenbrock function.



(A) sample size: 2 (B) sample size: 50

FIGURE 2. Convergence time of Rosenbrock function with different optimizers on different sample sizes.

3.1.1. *Convergence vs iterations:* The nonconvexity of this function makes the gradient-based algorithms converge slower than Newton/Quasi-Newton methods as can be seen in Figure 1. Another observation is the sharp drop in gradient in Newton/Quasi-Newton methods, indicating quadratic convergence. This is due to the methods adapting to the Hessian (or its approximation).

3.1.2. *Convergence vs time:* The Newton method had variable time for different sections of iteration, while the more adaptive Quasi-Newton methods had near-linear time. Observations are plotted in Figure 2

3.2. **Logistic Regression function.**

The logistic function was evaluated using the 4 methods with different regularization parameters, $\lambda = 0.001, 0.01, 0.1$.

3.2.1. *Convergence vs iterations:* Logistic regression is a convex function, which means that Newton's method adapts naturally. This is clearly visible in the Figure-3, where Newton has the shortest iteration count. Improvement in convergence is observed by lowering the value of λ. Hessian of logistic regression is given, by $AA^T(\sigma(A^T)\sigma(1-A^T))+\lambda I$. When λ is large, the Hessian is dominated by λ and the conditioning gets worse.

3.2.2. *Convergence vs time:* Figure-4 compares the time taken to converge vs. number of steps. The Newton converges faster in terms of steps taken, but each step in itself is computationally expensive, so the total time taken to converge is comparatively higher.

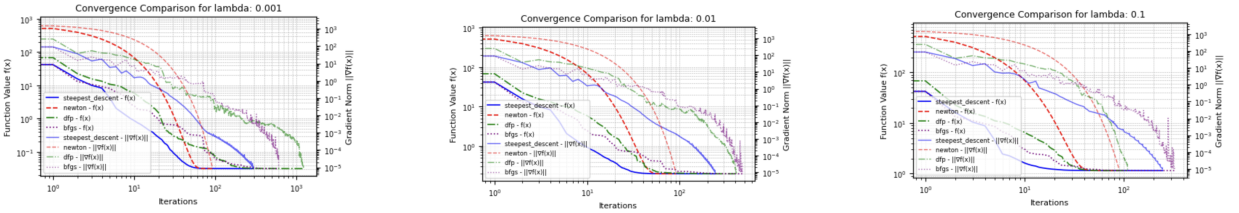


FIGURE 3. Steps to converge Logistic Regression function with different optimizers.

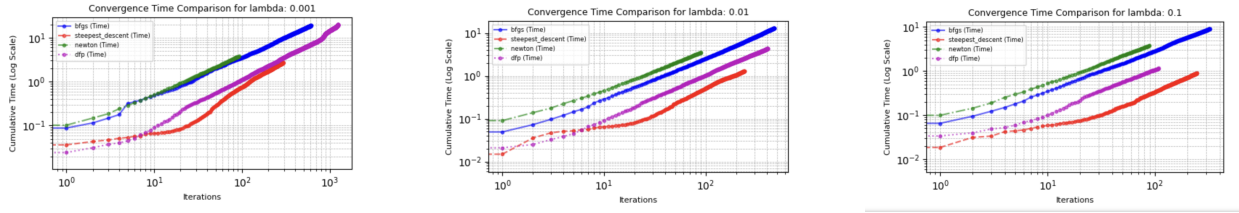
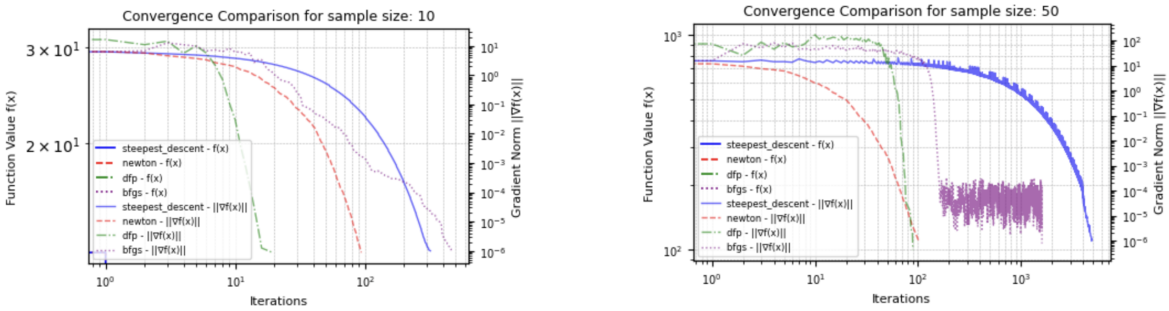


FIGURE 4. Time taken to converge Logistic function with different optimizers.



(A) sample size: 10 (B) sample size: 50

FIGURE 5. Convergence of Trid function has high correlation to sample size.

4. SUMMARY AND CONCLUSIONS

The key takeaway from the above analysis is that optimizers must be chosen based on the nature of the function.

- Picking the right Wolfe condition parameters is crucial for line search. For ex, the delta between parameters c_0 and c_1 had to be very less for DFP compared to other algorithms.
- The non-Lipschitz gradient of Rosenbrock causes the gradient based method to converge slower while the Hessian based methods performed well.
- While DFP performed well on Rosenbrock function with quadratic loss, it took longer to converge on sigmoidal loss function of Logistic Regression.

In order to correlate the sample size with convergence, I tested the optimizers on the Trid function. The variables of Trid function have strong dependency on its neighbors. Optimization gets challenging as changes to one variable affects the others. This can be confirmed with the visible zig-zagging in steepest descent and BFGS for higher dimensions in Figure-5

Acknowledgements

The author is thankful to Prof.Bamdad Hosseini for detailed discussions on Optimization algorithms.